Departamento de
Tecnología Digital

Tecsup TECNOLOGÍA CON SENTIDO



LABORATORIO N° 04

Desarrollo de aplicaciones Web: Implementación de Persistencia.

DOCENTE:

Coello Palomino, Ricardo

CURSO:

Desarrollo de aplicaciones Web
4 - C24 - Sección A - B

DESARROLLO DE APLICACIONES WEB AVANZADO: IMPLEMENTACIÓN DE PERSISTENCIA

I. Capacidades

El estudiante implementa y gestiona relaciones entre entidades utilizando ORM Hibernate en Spring Boot, aplicando correctamente las anotaciones @OneToOne y @ManyToMany, asegurando la integridad de los datos y el adecuado manejo de la persistencia en aplicaciones web.

II. Seguridad

- En este laboratorio está prohibida la manipulación del hardware, conexiones eléctricas o de red. Así como la ingesta de alimentos y bebidas.
- Ubicar maletines y/o mochilas en lugar destinado para tal fin.
- Dejar la mesa de trabajo y la silla utilizada limpias y ordenadas.

III. Fundamento teórico

- Revise el material de la semana correspondiente antes del desarrollo del laboratorio.

IV. Normas empleadas

- *No aplica*

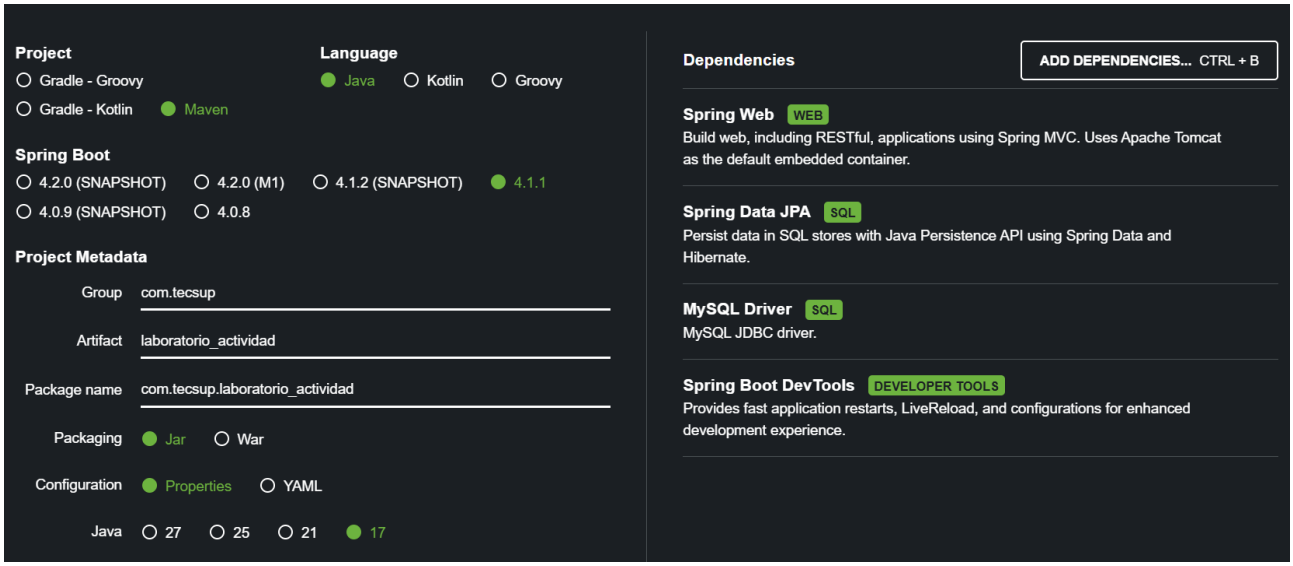
V. Recursos

- En este laboratorio, cada estudiante trabajará con una computadora con Windows 10.
- La instalación del software requerido se realizará en el equipo virtual.

VI. Metodología para el desarrollo de la tarea

- El desarrollo del laboratorio es individual.

VII. Procedimiento



EJERCICIO 1: RELACIÓN @OneToOne

Enunciado: Sistema de Usuario y Perfil

Una empresa desea desarrollar un sistema donde cada **usuario** tenga un único **perfil** asociado con información adicional.

Reglas del negocio:

- Un **Usuario** tiene:
  - id
  - username
  - password

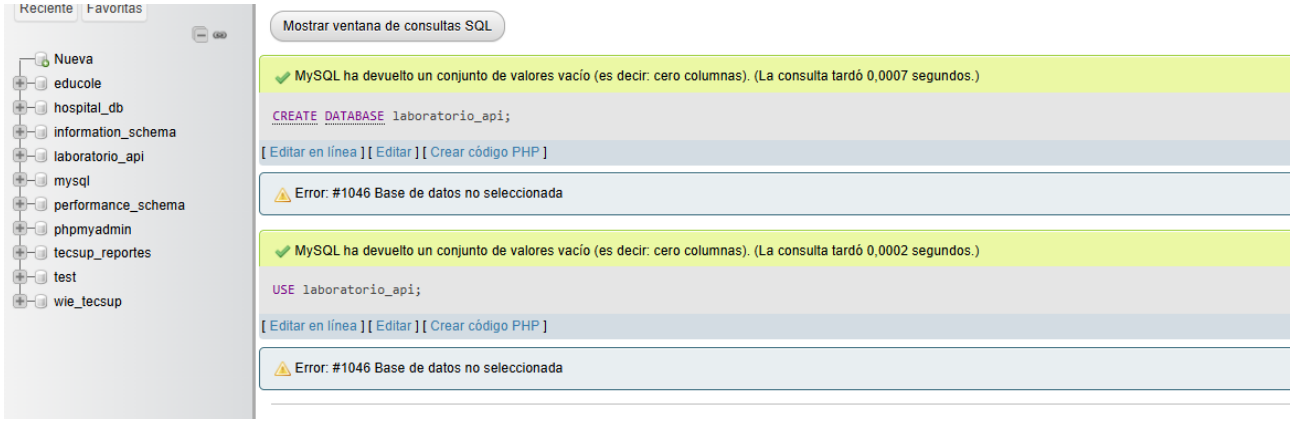
- Un **Perfil** tiene:
  - id
  - nombreCompleto
  - direccion
  - telefono
- Un usuario tiene **un solo perfil**.
- Un perfil pertenece a **un solo usuario**.

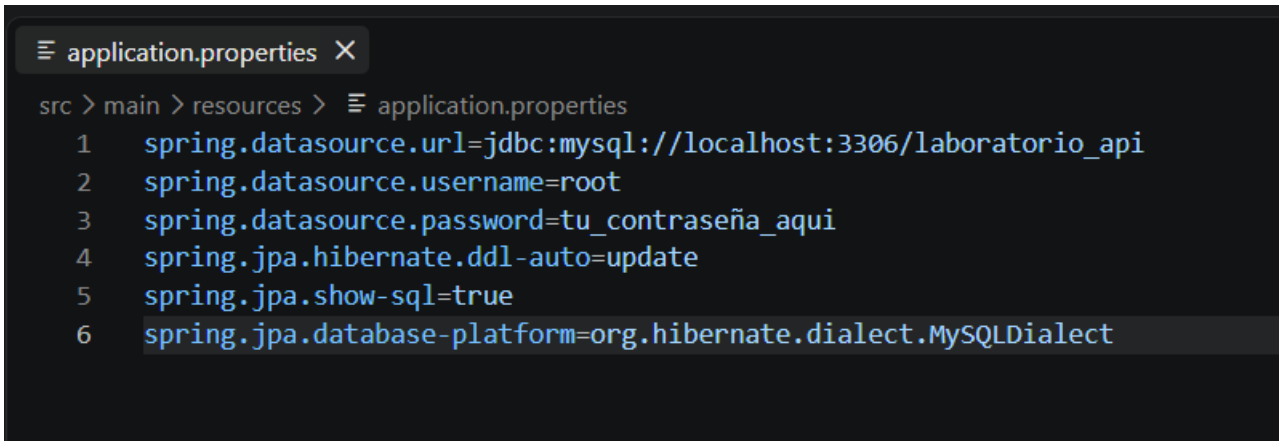
Actividades a desarrollar:

1. Crear las entidades:
   - Usuario

- Perfil

2. Implementar la relación:
   - Usar **@OneToOne**
   - Definir correctamente el **@JoinColumn**
3. Crear:
   - Repositories
   - Services
   - Controllers
4. Implementar endpoints:
   - Registrar usuario con perfil
   - Listar usuarios con su perfil
   - Obtener usuario por ID
   - Eliminar usuario (considerar cascada)





Usuario.java

```
package com.tecsup.laboratorio_actividad.models;



import jakarta.persistence.*;



@Entity

@Table(name = "usuario")

public class Usuario {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String username;

    private String password;



    @OneToOne(cascade = CascadeType.ALL)

    @JoinColumn(name = "perfil_id", referencedColumnName = "id")

    private Perfil perfil;



    // Getters y Setters

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }
```

```
    public String getUsername() { return username; }

    public void setUsername(String username) { this.username = username; }

    public String getPassword() { return password; }

    public void setPassword(String password) { this.password = password; }

    public Perfil getPerfil() { return perfil; }

    public void setPerfil(Perfil perfil) { this.perfil = perfil; }

}
```

Perfil.java

```
package com.tecsup.laboratorio_actividad.models;



import jakarta.persistence.*;



@Entity

@Table(name = "perfil")

public class Perfil {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String nombreCompleto;
```

```
    private String direccion;

    private String telefono;



    // Getters y Setters

    public Long getId() { return id; }

    public void setId(Long id) { this.id = id; }

    public String getNombreCompleto() { return nombreCompleto; }

    public void setNombreCompleto(String nombreCompleto) { this.nombreCompleto = nombreCompleto;
}

    public String getDireccion() { return direccion; }

    public void setDireccion(String direccion) { this.direccion = direccion; }

    public String getTelefono() { return telefono; }

    public void setTelefono(String telefono) { this.telefono = telefono; }

}
```

UsuarioController.java

```
package com.tecsup.laboratorio_actividad.controllers;



import com.tecsup.laboratorio_actividad.models.Usuario;

import com.tecsup.laboratorio_actividad.service.UsuarioService;
```

```
import org.springframework.web.bind.annotation.*;

import java.util.List;



@RestController

@RequestMapping("/api/usuarios")

public class UsuarioController {



    private final UsuarioService service;



    // Inyección por constructor (Reemplaza a @Autowired)

    public UsuarioController(UsuarioService service) {

        this.service = service;

    }



    @PostMapping

    public Usuario registrar(@RequestBody Usuario usuario) { return service.registrar(usuario); }



    @GetMapping

    public List<Usuario> listar() { return service.listar(); }
```

```
    @GetMapping("/{id}")
    public Usuario obtener(@PathVariable Long id) { return service.obtenerPorId(id); }

    @DeleteMapping("/{id}")
    public void eliminar(@PathVariable Long id) { service.eliminar(id); }
}
```

UsuarioService.java

```
package com.tecsup.laboratorio_actividad.service;

import com.tecsup.laboratorio_actividad.models.Usuario;
import com.tecsup.laboratorio_actividad.repository.UsuarioRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class UsuarioService {

    private final UsuarioRepository repo;
```

```
    // Inyección por constructor (Quita la advertencia de @Autowired)
    public UsuarioService(UsuarioRepository repo) {
        this.repo = repo;
    }

    public Usuario registrar(Usuario usuario) { return repo.save(usuario); }

    public List<Usuario> listar() { return repo.findAll(); }

    public Usuario obtenerPorId(Long id) {
        return repo.findById(id).orElseThrow(() -> new RuntimeException("Usuario no encontrado"));
    }

    public void eliminar(Long id) { repo.deleteById(id); }
}
```

EJERCICIO 2: RELACIÓN @ManyToMany

Enunciado: Sistema de Cursos y Estudiantes

Una institución educativa necesita gestionar los cursos y los estudiantes inscritos.

Reglas del negocio:

- Un **Estudiante** puede inscribirse en muchos cursos.
- Un **Curso** puede tener muchos estudiantes.

Entidades:

Estudiante:

- id
- nombre
- email

Curso:

- id
- nombre
- creditos

Actividades a desarrollar:

1. Crear entidades:
    - Estudiante
    - Curso
2. Implementar relación:
    - Usar **@ManyToMany**
    - Crear tabla intermedia:
        - estudiante_curso
3. Configurar:
    - @JoinTable
    - joinColumns y inverseJoinColumns
4. Crear:
    - Repositories
    - Services

- Controllers

5. Endpoints:
   - Crear estudiante con cursos
   - Listar estudiantes con sus cursos
   - Agregar curso a estudiante existente
   - Quitar cursos
   - Listar cursos de un estudiante
6. Validar:
   - Validar que no se repita la inscripción

Nota: Presentar capturas de pantalla del código y pruebas con Postman.

Estudiante.java

```
package com.tecsup.laboratorio_actividad.models;
import jakarta.persistence.*;
import java.util.ArrayList;
import java.util.List;

@Entity
@Table(name = "estudiante")
public class Estudiante {
   @Id
   @GeneratedValue(strategy = GenerationType.IDENTITY)
   private Long id; //[cite: 1]
   private String nombre; //[cite: 1]
   private String email; //[cite: 1]

   @ManyToMany //[cite: 1]
   @JoinTable(
      name = "estudiante_curso", //[cite: 1]
      joinColumns = @JoinColumn(name = "estudiante_id"), //[cite: 1]
      inverseJoinColumns = @JoinColumn(name = "curso_id") //[cite: 1]
   )
   private List<Curso> cursos = new ArrayList<>();

   // Getters y Setters
   public Long getId() { return id; }
   public void setId(Long id) { this.id = id; }
   public String getNombre() { return nombre; }
   public void setNombre(String nombre) { this.nombre = nombre; }
   public String getEmail() { return email; }
   public void setEmail(String email) { this.email = email; }
   public List<Curso> getCursos() { return cursos; }
   public void setCursos(List<Curso> cursos) { this.cursos = cursos; }
}
```

Curso.java

```
package com.tecsup.laboratorio_actividad.models;
import com.fasterxml.jackson.annotation.JsonIgnore;
import jakarta.persistence.*;
import java.util.List;

@Entity
@Table(name = "curso")
public class Curso {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id; //[cite: 1]
    private String nombre; //[cite: 1]
    private Integer creditos; //[cite: 1]

    @ManyToMany(mappedBy = "cursos")
    @JsonIgnore // Evita bucle infinito en JSON
    private List<Estudiante> estudiantes;

    // Getters y Setters
    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }
    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }
    public Integer getCreditos() { return creditos; }
    public void setCreditos(Integer creditos) { this.creditos = creditos; }
    public List<Estudiante> getEstudiantes() { return estudiantes; }
    public void setEstudiantes(List<Estudiante> estudiantes) { this.estudiantes = estudiantes; }
}
```

EstudianteService.java

```
package com.tecsup.laboratorio_actividad.service;

import com.tecsup.laboratorio_actividad.models.Curso;
import com.tecsup.laboratorio_actividad.models.Estudiante;
import com.tecsup.laboratorio_actividad.repository.CursoRepository;
import com.tecsup.laboratorio_actividad.repository.EstudianteRepository;
import org.springframework.stereotype.Service;
import java.util.List;

@Service
public class EstudianteService {

    private final EstudianteRepository estudianteRepo;
    private final CursoRepository cursoRepo;

    // Inyección por constructor (Reemplaza a @Autowired)
    public EstudianteService(EstudianteRepository estudianteRepo, CursoRepository cursoRepo) {
        this.estudianteRepo = estudianteRepo;
```

```
        this.cursoRepo = cursoRepo;
    }

    public Estudiante registrarConCursos(Estudiante estudiante) { return estudianteRepo.save(estudiante);
}

    public List<Estudiante> listarTodos() { return estudianteRepo.findAll(); }

    public Estudiante agregarCursoAEstudiante(Long estudianteId, Long cursoId) {
        Estudiante est = estudianteRepo.findById(estudianteId)
                .orElseThrow(() -> new RuntimeException("Estudiante no hallado"));
        Curso curso = cursoRepo.findById(cursoId)
                .orElseThrow(() -> new RuntimeException("Curso no hallado"));

        boolean yaInscrito = est.getCursos().stream().anyMatch(c -> c.getId().equals(cursoId));
        if(yaInscrito) {
            throw new RuntimeException("El estudiante ya está inscrito en este curso");
        }
        est.getCursos().add(curso);
        return estudianteRepo.save(est);
    }

    public Estudiante quitarCurso(Long estudianteId, Long cursoId) {
        Estudiante est = estudianteRepo.findById(estudianteId).orElseThrow();
        est.getCursos().removeIf(c -> c.getId().equals(cursoId));
        return estudianteRepo.save(est);
    }

    public List<Curso> listarCursosDeEstudiante(Long estudianteId) {
        return estudianteRepo.findById(estudianteId).orElseThrow().getCursos();
    }
}
```

EstudianteController.java

```
package com.tecsup.laboratorio_actividad.controllers;

import com.tecsup.laboratorio_actividad.models.Curso;
import com.tecsup.laboratorio_actividad.models.Estudiante;
import com.tecsup.laboratorio_actividad.service.EstudianteService;
import org.springframework.web.bind.annotation.*;
import java.util.List;

@RestController
@RequestMapping("/api/estudiantes")
public class EstudianteController {

    private final EstudianteService service;

    // Inyección por constructor (Reemplaza a @Autowired)
    public EstudianteController(EstudianteService service) {
```

```
        this.service = service;
    }

    @PostMapping
    public Estudiante registrar(@RequestBody Estudiante estudiante) {
        return service.registrarConCursos(estudiante);
    }

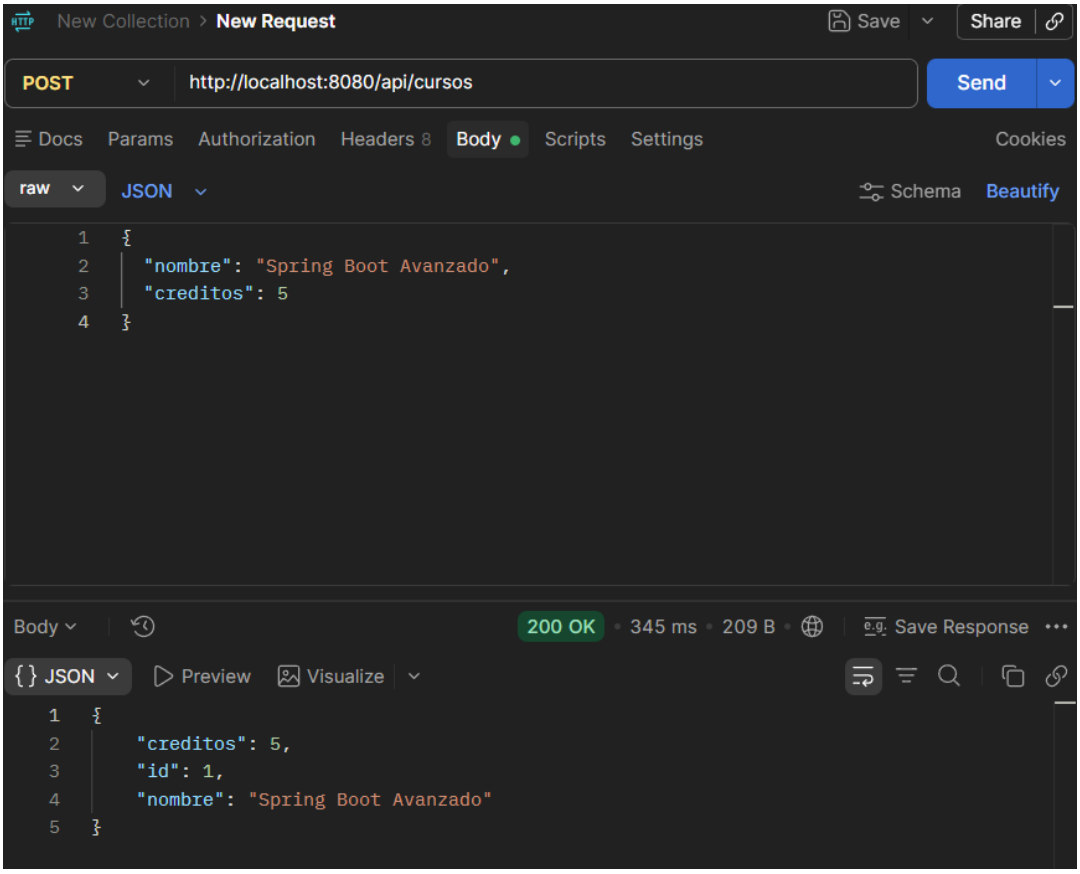
    @GetMapping
    public List<Estudiante> listar() {
        return service.listarTodos();
    }

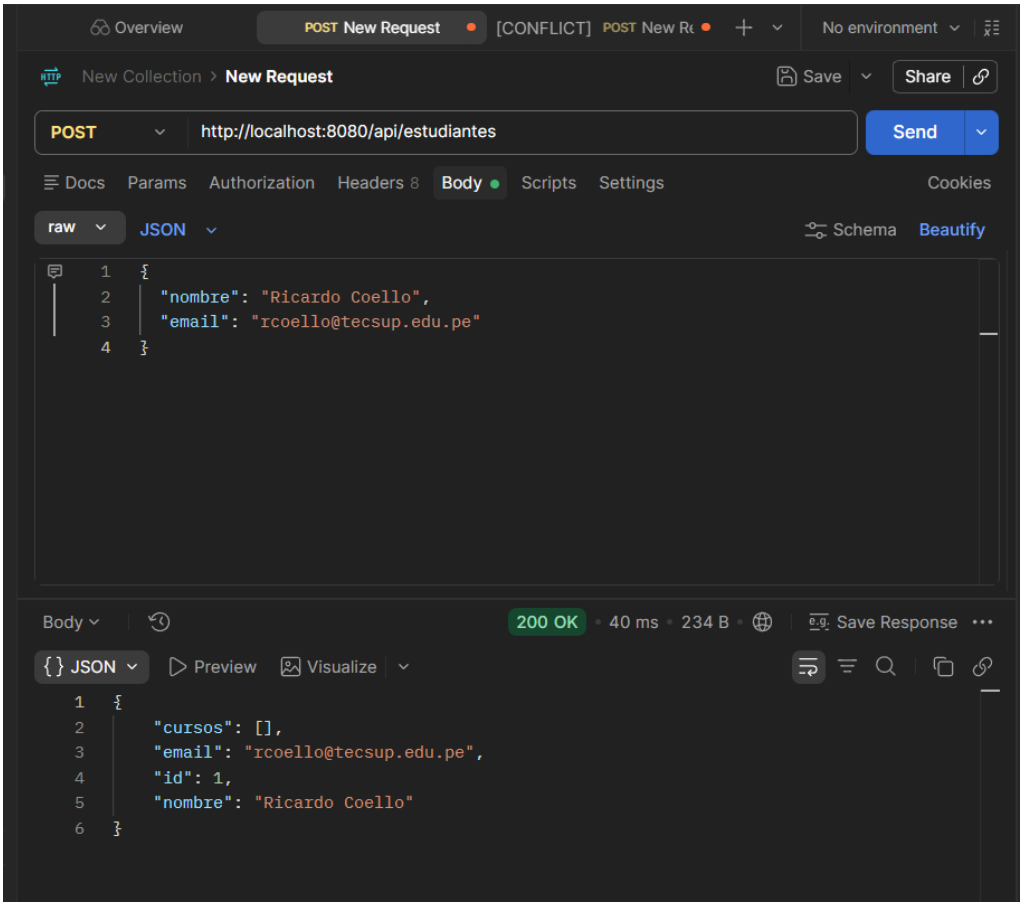
    @PutMapping("/{estudianteId}/cursos/{cursoId}")
    public Estudiante agregarCurso(@PathVariable Long estudianteId, @PathVariable Long cursoId) {
        return service.agregarCursoAEstudiante(estudianteId, cursoId);
    }

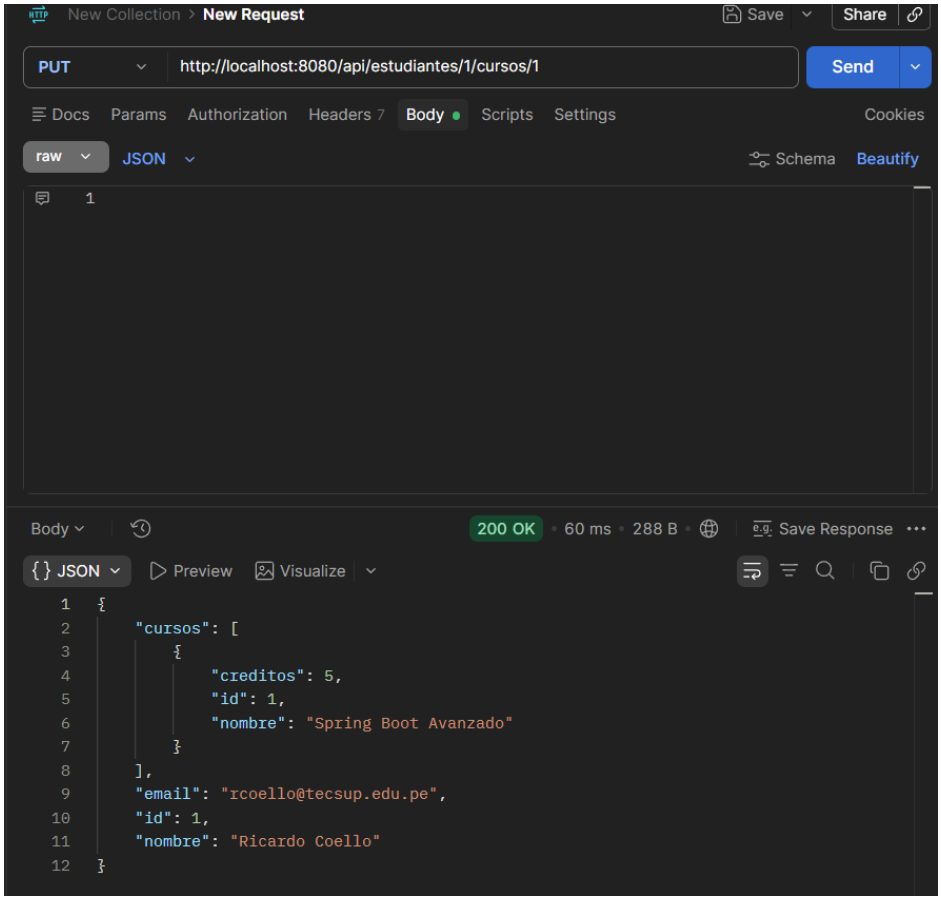
    @DeleteMapping("/{estudianteId}/cursos/{cursoId}")
    public Estudiante quitarCurso(@PathVariable Long estudianteId, @PathVariable Long cursoId) {
        return service.quitarCurso(estudianteId, cursoId);
    }

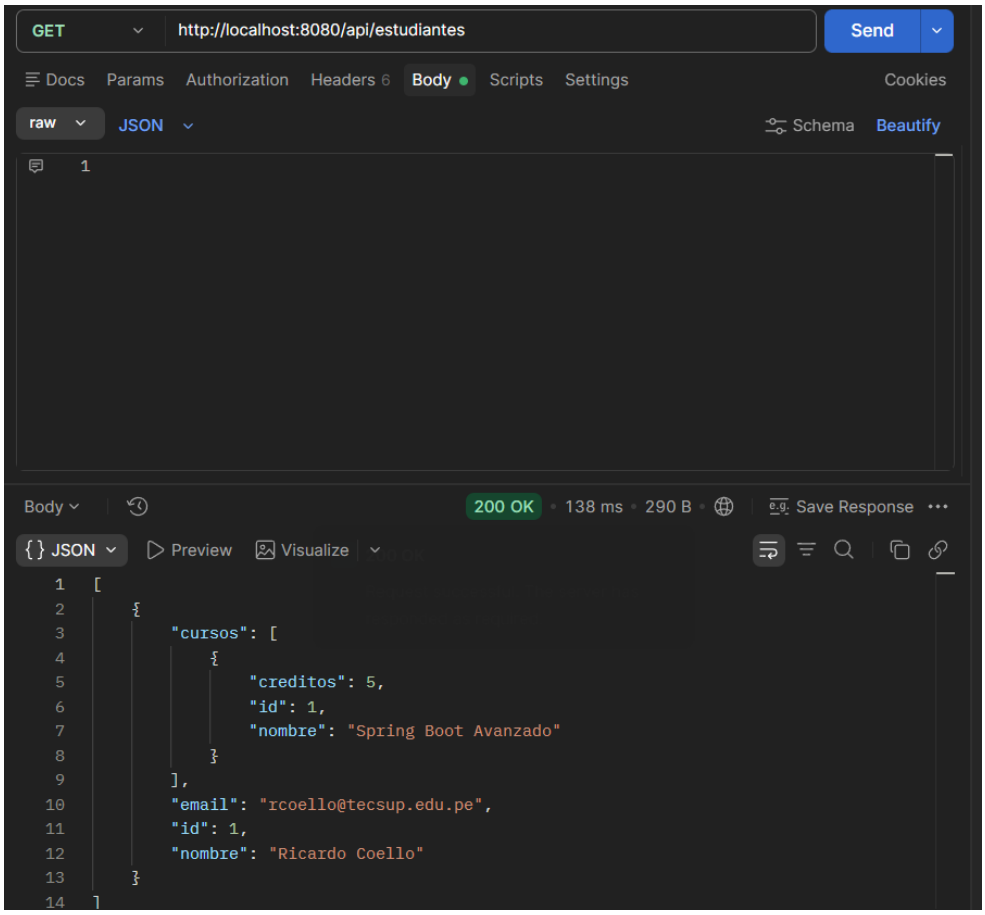
    @GetMapping("/{estudianteId}/cursos")
    public List<Curso> listarCursos(@PathVariable Long estudianteId) {
        return service.listarCursosDeEstudiante(estudianteId);
    }
}
```

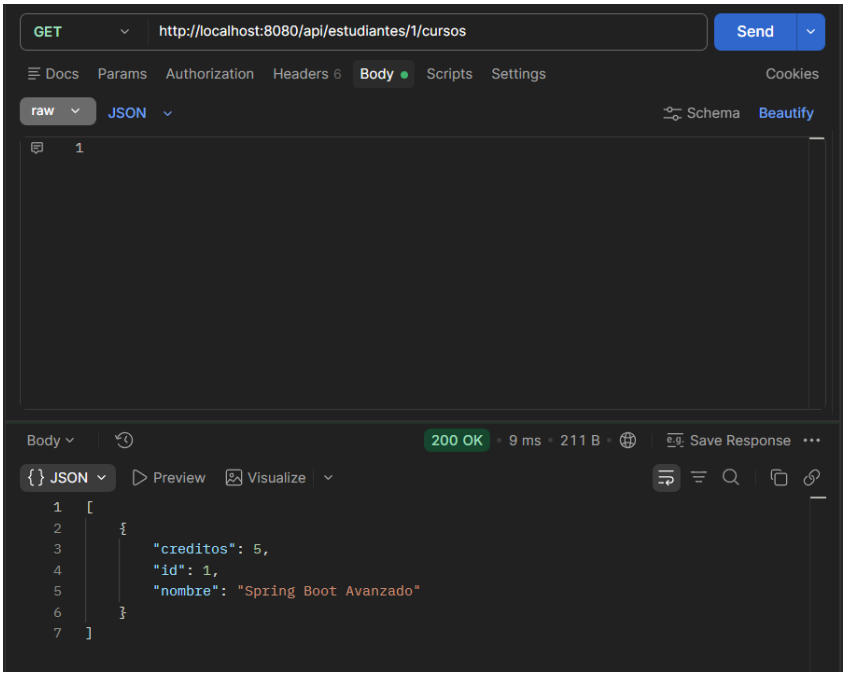
POSTMAN

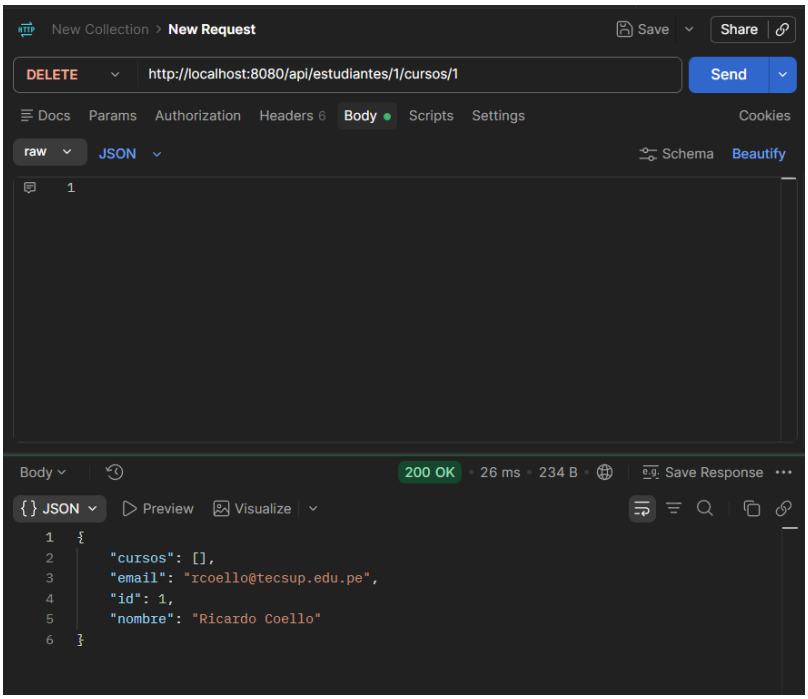

New Collection
New Request
Save
Share
POST
http://localhost:8080/api/cursos
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"nombre": "Spring Boot Avanzado",
"creditos": 5
}
Body
200 OK
345 ms
209 B
Save Response
JSON
Preview
Visualize
{
"creditos": 5,
"id": 1,
"nombre": "Spring Boot Avanzado"
}

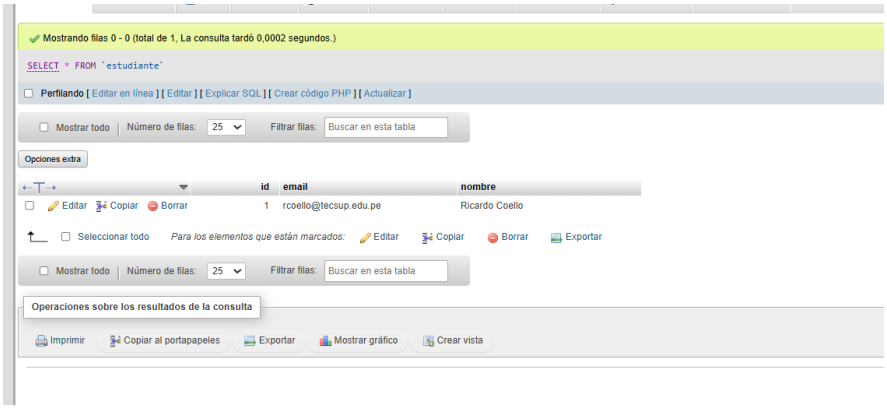

Overview
POST New Request
[CONFLICT] POST New Re
No environment
New Collection
New Request
Save
Share
POST
http://localhost:8080/api/estudiantes
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
{
"nombre": "Ricardo Coello",
"email": "rcoello@tecsup.edu.pe"
}
Body
200 OK
40 ms
234 B
Save Response
JSON
Preview
Visualize
{
"cursos": [],
"email": "rcoello@tecsup.edu.pe",
"id": 1,
"nombre": "Ricardo Coello"
}


New Collection > New Request
Save
Share
PUT
http://localhost:8080/api/estudiantes/1/cursos/1
Send
Docs
Params
Authorization
Headers 7
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
1
Body
200 OK
60 ms
288 B
Save Response
{ } JSON
Preview
Visualize
{
"cursos": [
{
"creditos": 5,
"id": 1,
"nombre": "Spring Boot Avanzado"
}
],
"email": "rcoello@tecsup.edu.pe",
"id": 1,
"nombre": "Ricardo Coello"
}


GET
http://localhost:8080/api/estudiantes
Send
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
Cookies
raw
JSON
Schema
Beautify
1
Body
200 OK
138 ms
290 B
Save Response
{ } JSON
Preview
Visualize
[
{
"cursos": [
{
"creditos": 5,
"id": 1,
"nombre": "Spring Boot Avanzado"
}
],
"email": "rcoello@tecsup.edu.pe",
"id": 1,
"nombre": "Ricardo Coello"
}
]


GET
http://localhost:8080/api/estudiantes/1/cursos
Send
Docs Params Authorization Headers 6 Body Scripts Settings Cookies
raw JSON
Schema Beautify
1
Body
200 OK 9 ms 211 B
Save Response
{} JSON Preview Visualize
[
{
"creditos": 5,
"id": 1,
"nombre": "Spring Boot Avanzado"
}
]


New Collection › New Request
Save
Share
DELETE
http://localhost:8080/api/estudiantes/1/cursos/1
Send
Docs Params Authorization Headers 6 Body Scripts Settings Cookies
raw JSON
Schema Beautify
1
Body
200 OK 26 ms 234 B
Save Response
{} JSON Preview Visualize
{
"cursos": [],
"email": "rcoello@tecsup.edu.pe",
"id": 1,
"nombre": "Ricardo Coello"
}


Mostrando filas 0 - 0 (total de 1, La consulta tardó 0,0002 segundos.)
SELECT * FROM `estudiante`
Perfilando [Editar en línea] [Editar] [Explicar SQL] [Crear código PHP] [Actualizar]
Mostrar todo Número de filas: 25 Filtrar filas: Buscar en esta tabla
Opciones extra
id email nombre
Editar Copiar Borrar 1 rcoello@tecsup.edu.pe Ricardo Coello
Seleccionar todo Para los elementos que están marcados: Editar Copiar Borrar Exportar
Mostrar todo Número de filas: 25 Filtrar filas: Buscar en esta tabla
Operaciones sobre los resultados de la consulta
Imprimir Copiar al portapapeles Exportar Mostrar gráfico Crear vista

OBSERVACIONES:

Error de sintaxis en el IDE por funciones autocompletadas: Durante el desarrollo, al usar funciones como orElseThrow(), el editor (Visual Studio Code) agregó visualmente la etiqueta de parámetro message:. Copiar y pegar esto literalmente generó un error de compilación que debió ser borrado manualmente.

Advertencias de inyección de dependencias: El entorno de desarrollo mostró múltiples advertencias amarillas al utilizar la anotación @Autowired para inyectar los repositorios. Se observó que la buena práctica actual en Spring Boot sugiere reemplazar esto por inyección mediante constructores (Constructor Parameter).

Conflictos por estructura de paquetes (Packages): Se presentó un error crítico donde el IDE no reconocía las clases (Usuario, Perfil) debido a que la declaración de los paquetes en la primera línea del código (package com.tecsup.models;) no coincidía con la ruta real del proyecto (com.tecsup.laboratorio_actividad.models).

Clases duplicadas en diferentes carpetas: Se identificó un problema de compilación generado al guardar accidentalmente una copia de la clase UsuarioService.java dentro de la carpeta repository. El proyecto no podía compilar correctamente hasta que el archivo redundante fue eliminado.

Error de acceso a la base de datos MySQL (Access Denied): Al ejecutar la aplicación por primera vez, la consola de Spring Boot devolvió el error Access denied for user 'root'@'localhost' (using password: YES). Esto ocurrió porque el archivo application.properties tenía una contraseña temporal definida, mientras que el servicio de XAMPP local usa el usuario "root" sin contraseña.

CONCLUSIONES:

Gestión de relaciones @OneToOne: Aprendí a implementar correctamente una relación bidireccional uno a uno entre Usuario y Perfil. Comprendí la importancia del atributo cascade = CascadeType.ALL para asegurar que las operaciones (como eliminar un usuario) afecten automáticamente al perfil asociado en la base de datos.

Implementación y control de relaciones @ManyToMany: Logré configurar la relación de muchos a muchos entre Estudiante y Curso utilizando @JoinTable. Esto me permitió entender cómo JPA crea automáticamente la tabla intermedia (estudiante_curso) sin necesidad de modelarla como una entidad separada en Java.

Prevención de bucles infinitos en JSON: Durante el desarrollo del Ejercicio 2, comprobé que al devolver respuestas JSON en relaciones bidireccionales se puede generar un bucle infinito. Resolví este problema aplicando de manera efectiva la anotación @JsonIgnore en la lista de estudiantes dentro de la clase Curso.

Validaciones lógicas en la capa Service: Logré implementar con éxito una validación personalizada dentro de EstudianteService (usando .stream().anyMatch()) para evitar que un estudiante pueda matricularse dos veces en un mismo curso, demostrando cómo interceptar errores lógicos antes de persistir los datos.

Pruebas de la API REST con Postman: A través de este laboratorio, consolidé mi capacidad para testear endpoints. Comprobé exitosamente operaciones CRUD complejas mediante peticiones POST para crear entidades anidadas, peticiones PUT para actualizar tablas intermedias (matricular) y verificando los códigos de estado HTTP (como el error 500 para la inscripción duplicada).